University of Bologna
Department of Industrial Engineering

ARGOS Laboratory
Autonomous Robust Guidance and Optimization for Spaceflight

RAPTOR

Rapid Algorithm for Pseudospectral Trajectory Optimization and Robustification



User Manual

Version 0.1.0 • June 2026

Author: Riccardo Minnozzi
Contact: riccardo.minnozzi@unibo.it

Contents

List of Acronyms **1**

1 Introduction **1**
1.1 Purpose of This Manual . . . 1
1.2 What Is RAPTOR? . . . 1
1.3 Conventions Used in This Manual . . . 1

2 Installation **2**
2.1 System Requirements . . . 2
2.2 Installing RAPTOR . . . 2
2.3 Configuring Third-Party Solvers . . . 2
2.4 Verifying the Installation . . . 2

3 Getting Started **3**
3.1 Anatomy of an Example . . . 3
3.2 Creating Your First Problem . . . 4
3.3 Typical Workflow . . . 4
3.4 Working Interactively . . . 5

4 Defining an Optimal Control Problem **5**
4.1 Problem Object . . . 5
4.2 Phases . . . 6
4.3 Variable Bounds . . . 6
4.4 Fixed Boundary Values . . . 6
4.5 Initial Guess . . . 7
4.6 User-Defined Functions . . . 7
4.6.1 Path Functions . . . 7
4.6.2 Boundary Functions . . . 8
4.6.3 Dynamics . . . 8
4.6.4 Constraints . . . 8
4.6.5 Objective Functions . . . 9
4.6.6 Function Properties and Derivatives . . . 9
4.7 Collocation Settings . . . 9
4.8 Linkage Conditions . . . 10

5 Configuring the Solver **10**
5.1 Configuration Order . . . 10
5.2 Scaling . . . 10
5.3 Numerical Differentiation . . . 11
5.4 NLP Solvers . . . 11
5.5 Solution Validation . . . 11
5.6 Postprocessing and Plotting . . . 12

6 Solving the Problem **12**
6.1 Building the Transcribed Problem . . . 12
6.2 Running the NLP Solver . . . 12
6.3 Accessing the Solution . . . 13
6.4 Plotting Results . . . 13
6.4.1 Automatic Plot Metadata . . . 13
6.4.2 Custom Plotting Callbacks . . . 13
6.5 Diagnosing Problems . . . 14
6.5.1 Build Errors . . . 14

6.5.2 Failure to Converge 14
6.5.3 Converged but Inaccurate Solutions 14
6.5.4 Requesting Support 14

List of Acronyms

RAPTOR Rapid Algorithm for Pseudospectral Trajectory Optimization and Robustification

NLP Nonlinear Programming

1 Introduction

1.1 Purpose of This Manual

This manual is a practical guide to installing and using the Rapid Algorithm for Pseudospectral Trajectory Optimization and Robustification (RAPTOR) package to formulate and solve optimal control problems. It introduces the standard workflow, explains the main user-facing components, and provides an initial troubleshooting reference.

The manual is intended for both new and experienced users. It assumes basic familiarity with MATLAB and optimal control terminology, but no prior knowledge of RAPTOR is required. New users should read the sections in order; users diagnosing a problem can proceed directly to the relevant configuration or troubleshooting section.

The emphasis is on practical use of the toolbox. Derivations of the underlying algorithms and implementation details of the numerical methods are outside the scope of this manual and are described in the accompanying scientific publications.

1.2 What Is RAPTOR?

Rapid Algorithm for Pseudospectral Trajectory Optimization and Robustification is an object-oriented MATLAB toolbox for solving continuous-time optimal control problems using pseudospectral methods. It provides a modular interface for defining a problem, configuring its transcription and optimization settings, solving the resulting Nonlinear Programming (NLP), and examining the computed solution. The interface is intended to keep problem formulations readable while retaining the flexibility required for research applications.

RAPTOR is under active development. Interfaces, available options, and supported third-party solvers may therefore change between releases. Consult the release notes before reusing a problem definition created with an earlier version. The toolbox is available at , and its documentation is available at .

Git repository URL

documen URL

1.3 Conventions Used in This Manual

The following conventions are used throughout this manual:

- Teletype font identifies `variables`, `functions`, `classes`, and `methods`.
- Sans-serif font identifies files and folders/.
- MATLAB code is presented as follows:

Example Code Snippet

```
% this is an example
exampleVariable = exampleFunction();
```

- Command-line instructions are presented as follows:

```
C:\path\to\folder> cd
```

Mathematical quantities follow a column-vector convention throughout the toolbox. A time-dependent quantity is represented by a matrix in which each row corresponds to one component and each column corresponds to one evaluation time. For example, a state vector $\mathbf{x}(t)$ with n_x components evaluated at N time points is represented by an $n_x \times N$ matrix. This convention is particularly important when implementing vectorized dynamics, constraints, and cost functions.

2 Installation

This section explains how to install RAPTOR, configure its external dependencies, and verify that the package can build and solve an example problem.

2.1 System Requirements

RAPTOR requires a working MATLAB installation. The package has been tested with MATLAB R2023b, R2025b, and R2026a. Compatibility with other releases is not guaranteed. When reporting a compatibility issue, include the MATLAB release, operating system, RAPTOR version, and complete error message.

RAPTOR relies on third-party software to solve the transcribed NLP. IPOPT [1] is provided as a precompiled binary for Windows, macOS, and Linux through the `https://github.com/ebertolazzi/mexIPOPT` interface. SNOPT [**gill2002˙SnoptSqpAlgorithm**] and WORHP [**wassel2013˙ExploringNovelDesig** are also supported but must be installed separately. See subsection 2.3 for solver-specific installation and configuration requirements.

2.2 Installing RAPTOR

Download or clone the RAPTOR repository into a folder where you have write permission. To clone the repository, run:

```
C:\path\to\folder> git clone <RAPTOR repository URL>
```

Placeholder: replace `<RAPTOR repository URL>` with the public repository URL before publication.

In MATLAB, change to the repository root and run the initialization function:

Adding RAPTOR to the MATLAB Path

```
cd C:\path\to\raptor
RAPTOR_init();
```

The initialization function modifies the MATLAB path only for the current session. Run it again after restarting MATLAB unless you explicitly save the updated path after installation.

2.3 Configuring Third-Party Solvers

In addition to the bundled IPOPT interface, RAPTOR supports SNOPT and WORHP. Each solver has its own installation, licensing, and MATLAB interface requirements. Install the solver according to its vendor documentation, then confirm that MATLAB can locate and call the interface before selecting it in RAPTOR. Installing external solver interfaces under third-party/solvers/ in the RAPTOR installation directory provides a consistent project layout. Solver selection is described in section 5.

Placeholder: provide separate setup instructions for SNOPT, WORHP, IPOPT, and every other supported back end. For each solver, include the supported version, license requirement, required environment variables, MATLAB path setup, a minimal standalone verification command, and the corresponding value of `problem.config.solver.method`.

2.4 Verifying the Installation

After initializing RAPTOR and configuring a solver, run a small example from the repository root:

Running Your First Example

```
% Move to the selected example.
cd examples/
cd 00-doubleInt/
main;
```

For a more complete verification, run the example test utility from the repository root. It executes all available examples and may take approximately two minutes, depending on the machine:

Running All Available Examples

```
% Move to the test folder.
cd tests/
run_all_examples();
```

A [PASS] result confirms that the corresponding example completed successfully in the current environment. Investigate each [ERROR] result individually. Such a result does not necessarily indicate a fault in the core package, because some examples may require a solver that is unavailable or unlicensed on the current machine.

When an example fails, first verify that MATLAB can locate the selected solver, then rerun only that example and retain the complete command-window output. If the failure persists, compare the example's solver configuration with the installation instructions in subsection 2.3.

3 Getting Started

Before implementing a new problem, work through at least one supplied example. The examples show the expected script structure, naming conventions, and relationship between problem definition, transcription, solver configuration, and postprocessing.

3.1 Anatomy of an Example

This section uses the brachistochrone problem in examples/03-brachistochrone/, based on [2], to illustrate the structure of a typical example. The same overall sequence is used throughout this manual.

The script begins by defining constants, boundary values, and initial-guess data. Constants required by user-defined dynamics, costs, or constraints are stored in an auxiliary-data structure so that they remain available during function evaluation.

Constants and Auxiliary Data

```
%% Constants
aux.g = 9.8; aux.a = 1; aux.b = 1.5;
x0 = 0; y0 = 0; v0 = 0; t0 = 0;
xf = 10; yf = 10;
tf_guess = 2; vf_guess = 50; u_guess = 0.5;
```

The script then creates the continuous optimal control problem through RAPTOR's object-oriented interface. It defines the phases, dimensions, functions, bounds, initial guess, and collocation settings. These elements are explained in section 4.

Problem Definition

```
%% Problem definition
% Start timing the build process.
tic

% Instantiate the problem object.
ocp = raptor.api.v1.OptimalControlProblem('brachistochrone', aux, 0);

% Add the single phase.
phase = ocp.addPhase('descent', 3, 1, 0);

[...]

% Set the collocation parameters.
```

```
phase.collocation.interpolant = 'Lagrange';
phase.collocation.polynomials = 'Legendre';
phase.collocation.nodes = 'Gauss-Radau';
phase.collocation.nNodes = 49;
```

The call to `raptor.build` checks the supplied definition and constructs the transcribed numerical problem. Configuration options can then be adjusted before calling the selected NLP solver. The configuration shown below is specific to the brachistochrone example; a new problem may require different scaling, differentiation, validation, and solver settings.

Solver Configuration and Solution

```
% Build the problem.
problem = raptor.build(ocp);

% Adjust the numerical configuration if needed.
problem.config.transcription.method = 'NLP';
problem.config.scaling.method = ['BoundsProjection' 'RuizStatic'];
problem.config.transcription.numericalJacobian = '
   ComplexStepDifferences';
problem.config.transcription.finiteStep = 1e-10;
problem.config.postprocessing.validation.method = 'RungeKutta45';
problem.config.solver.method = 'Snopt';
problem.config.solver.tol = 1e-8;
problem.config.postprocessing.plotting.primal = showPlots;
problem.config.postprocessing.plotting.dual = showPlots;
problem.config.postprocessing.plotting.validation = showPlots;
if showPlots
    problem.config.postprocessing.plotting.auxiliaryFunction = @
       visualizeBrachistochroneSolution;
end

% Solve the problem.
sol = problem.solve();
```

3.2 Creating Your First Problem

The `createMainTemplate` utility generates a main.m template in the current MATLAB folder. The generated script follows the structure used by the supplied examples. Complete and execute one section at a time so that MATLAB can expose available object properties through autocomplete and errors can be isolated early.

The utility overwrites an existing main.m file in the current folder. Run it in a new folder or rename any existing main.m file before calling it.

The template is a starting point rather than a required project layout. Once the formulation works, move user functions and reusable setup code into separate files to keep the main script concise. After becoming familiar with the interface, users can organize problem definitions in the way best suited to their application; a mature main script may contain only the calls required to create, build, configure, and solve the problem.

3.3 Typical Workflow

The typical RAPTOR workflow consists of the following steps:

1. write the continuous mathematical formulation and identify all states, controls, parameters, constraints, boundary conditions, and objective terms;

2. collect constants and auxiliary data required by user-defined functions;
3. create the problem and its phases as described in section 4;
4. provide bounds, boundary data, an initial guess, and collocation settings;
5. build the transcribed problem and resolve any definition errors;
6. configure scaling, differentiation, postprocessing, and the NLP solver as described in section 5;
7. solve the problem, inspect the solver status, and validate the resulting trajectory.

A successful build indicates that the supplied definition passed the checks performed by RAPTOR; it does not prove that the continuous problem is feasible, correctly modelled, or numerically well conditioned. If the solver fails, compare the implemented formulation with the original equations before changing numerical settings.

3.4 Working Interactively

When developing a problem interactively, execute the setup code incrementally. Keeping instantiated objects in the MATLAB workspace improves autocomplete support and helps identify the first line that introduces an invalid value. This is especially useful when exploring configuration properties or working inside MATLAB functions, where autocomplete may have less context.

After each major step, inspect the object that was modified before proceeding. In particular, verify phase dimensions after creation, bounds and boundary values before constructing the guess, and configuration fields after selecting a numerical method. This incremental approach makes definition errors easier to isolate than running a complete new formulation for the first time.

4 Defining an Optimal Control Problem

The RAPTOR interface represents an optimal control problem as a top-level problem object containing one or more phases. Each phase defines a time interval with its own variables, dynamics, constraints, cost terms, boundary data, initial guess, and discretization settings. A typical problem definition follows this order:

1. create the problem object;
2. add one or more phases with the required dimensions;
3. define variable bounds and fixed boundary values;
4. provide an initial guess;
5. register the dynamics, constraints, and cost terms;
6. select the collocation settings;
7. add linkage conditions when the problem contains multiple phases.

The examples in this section are excerpts from the brachistochrone problem. They illustrate individual parts of the interface and are not intended to form a complete executable script.

4.1 Problem Object

Create the top-level problem object before defining any phase. Its constructor receives a descriptive name, a structure containing auxiliary data, and the number of global parameters shared by all phases. The problem name is used for display purposes in automatically generated plots. Auxiliary data may contain physical constants, reference values, and other fixed quantities required by user-defined functions.

Optimal Control Problem

```
ocp = raptor.api.v1.OptimalControlProblem('brachistochrone', aux, 0);
```

Keep optimization variables out of the auxiliary-data structure. Auxiliary data is treated as fixed: RAPTOR neither differentiates with respect to it nor changes it during optimization. The final constructor argument is zero in this example because the brachistochrone problem has no global parameters.

4.2 Phases

A phase represents a time interval over which the model dimensions and governing equations remain unchanged. Introduce a new phase when the number of variables, dynamics, constraints, or operating regime changes at a known event. For example, examples/09-boostback2D/ uses separate phases for different flight regimes.

The `addPhase` method receives a unique phase name followed by the number of states, controls, and static phase parameters. These dimensions determine the expected sizes of the data passed to and returned by the phase functions. The phase name is also used to access phase-specific results in the solution structure.

Phase Definition

```
phase = ocp.addPhase('descent', 3, 1, 0);
```

The brachistochrone problem therefore contains one phase named `descent`, with three states, one control, and no static phase parameters.

4.3 Variable Bounds

Box constraints set component-wise lower and upper bounds on time, states, controls, and parameter variables. Bounds on global parameters are defined on the top-level problem object; bounds on phase variables are defined on the corresponding phase. Bounds should reflect the mathematical formulation and known physical limits. Realistic finite bounds can also improve scaling and restrict the solver's search region; scaling is discussed in section 5.

Variable Bounds

```
phase.box.lower.finalTime = 0.1;
phase.box.upper.finalTime = 5;
phase.box.lower.states = [0 0 -100];
phase.box.upper.states = [xf 100 100];
phase.box.lower.controls = 0;
phase.box.upper.controls = pi/2;
```

State, control, and parameter bounds must follow the component order used throughout the phase definition. During transcription, RAPTOR checks their dimensions and verifies that each lower bound does not exceed its corresponding upper bound.

4.4 Fixed Boundary Values

The `boundaryConditions` fields fix selected variables at the initial or final boundary of a phase. Assign only the components prescribed by the mathematical formulation; components that remain free should be left unset. Individual components may be selected by indexing the relevant field.

In the brachistochrone example, the initial time and all initial states are fixed, while only the first two final-state components are prescribed.

Boundary Conditions

```
phase.boundaryConditions.initial.time = t0;
```

```
phase.boundaryConditions.initial.states = [x0 y0 v0];
phase.boundaryConditions.final.states(1 : 2) = [xf yf];
```

A fixed boundary value must satisfy the corresponding variable bounds. A conflict between a boundary value and a box constraint makes the transcribed problem infeasible. General endpoint relations that cannot be expressed by fixing a variable directly should instead be implemented as boundary constraints, as described in 4.6.4.

4.5 Initial Guess

The initial guess provides the trajectory from which the NLP solver starts. A guess that respects the bounds and boundary conditions and approximates the expected trajectory shape generally improves convergence.

RAPTOR can construct a guess by linear interpolation or by numerical integration of the dynamics. For a linear guess, specified boundary values form the interpolation endpoints; where a boundary value is absent, the corresponding initial-guess value is used. For an integration-based guess, the initial time and states and the final time must be available from either the boundary conditions or the guess. Controls and static parameters are interpolated from the continuous guess data during integration. A single control value produces a constant profile, while two endpoint values produce a linear profile.

The brachistochrone example specifies selected terminal and continuous values, then calls `linear` to construct the remaining profiles.

Initial Guess

```
phase.initialGuess.final.time = tf_guess;
phase.initialGuess.final.states(3) = vf_guess;
phase.initialGuess.continuous.controls = u_guess;
phase.initialGuess.linear();
```

Inspect the generated guess before solving, particularly for incorrect units, discontinuities, and bound violations. Interior-point solvers may modify an infeasible initial guess before starting their iterations, which can obscure the source of unexpected behavior.

4.6 User-Defined Functions

Dynamics, constraints, cost terms, and custom linkage conditions are supplied as MATLAB function handles. RAPTOR passes the quantities available to each function through a single `inputData` object. User-defined functions fall into two categories according to where they are evaluated: path functions and boundary functions.

4.6.1 Path Functions

A path function is evaluated throughout a phase. This category includes dynamics, path and differential constraints, and Lagrange cost terms. The term *path function* therefore describes a common evaluation interface and should not be confused with the more specific concept of a path constraint.

Path functions receive continuous-time quantities through fields such as `inputData.time`, `inputData.states`, `inputData.controls`, and `inputData.parameters`, together with the auxiliary data associated with the problem. The available fields depend on the phase dimensions and the type of function being evaluated.

Every path function must support vectorized evaluation. The columns of each time-dependent input correspond to distinct evaluation points, and the returned value must contain the same number of columns. For example, if `inputData.time` contains N time points, the state and control inputs also contain N columns, and the function output must contain N columns.

4.6.2 Boundary Functions

A boundary function is evaluated using quantities at the endpoints of one or more phases. This category includes boundary constraints, Mayer cost terms, and custom linkage conditions. Boundary functions receive endpoint data through fields such as `inputData.initialTime`, `inputData.initialStates`, `inputData.initialControls`, `inputData.initialParameters`, and the corresponding `final...` fields, together with the auxiliary data available to the function.

Boundary functions are evaluated once for the relevant phase endpoints and do not use the vectorized continuous-time convention required by path functions. A custom linkage function follows the same boundary-function convention, but its final quantities refer to the first linked phase and its initial quantities refer to the second.

4.6.3 Dynamics

A dynamics function returns the state derivatives enforced throughout a phase. Its output must contain one row for each state, in the same order used when the phase was declared, and one column for each input time point. Register the function with the phase dynamics object using `add`.

Dynamics Registration

```
dyn = phase.dynamics.add(@brachistochroneDynamics);
```

In this example, `brachistochroneDynamics` implements the complete right-hand side of the state equations. If the dynamics are assembled from several additive contributions, each contribution may be registered separately as described in subsubsection 4.6.6.

4.6.4 Constraints

Constraints are organized by where and how they are enforced. A path constraint applies throughout a phase, a boundary constraint relates quantities at one or both phase endpoints, and a differential constraint is defined through the corresponding `differential` constraint family. Path and differential constraints use the path-function interface, whereas boundary constraints use the boundary-function interface.

For each constraint family, `stack` creates a named constraint group and assigns its lower and upper bounds. The group name must be unique within the phase. It is used both to access the constraint object during problem definition and to identify its values in the solution structure. The `add` method then registers one or more functions whose summed output defines that constraint group.

Path Constraints Path constraints impose equalities or inequalities at the continuous evaluation points of a phase. They are appropriate for limits derived from the instantaneous states, controls, or parameters.

The brachistochrone example constrains the local slope of the curve:

Path Constraint

```
phase.constraints.path.stack('slope', -inf, aux.b);
phase.constraints.path.slope.label = '$dy/dx$';
f = phase.constraints.path.slope.add(@brachistochroneSlopeConstraint)
   ;
f.functionType = 'linear';
```

Boundary Constraints Boundary constraints impose general equalities or inequalities involving initial and final quantities. Use them when an endpoint requirement depends on several variables, relates the initial and final boundaries, or must remain within an interval. This differs from assigning a fixed boundary value through `boundaryConditions`, which directly fixes one selected variable component to a known value. Note that a `boundaryCOndition` can also be interpreted as a specific instance of a boundary constraint.

Differential Constraints Differential constraints implement functions whose time derivative is to be constrained. These constraints are registered through `phase.constraints.differential`. They are kept separate from algebraic path constraints because their transcription requires the application of pseudospectral operators, but their functions follow the same vectorized path-function convention. Differential constraints can be used to enforce limits on the rates of change of specific quantities (e.g., the thrust rate limit implemented in examples/01-landing1D/

4.6.5 Objective Functions

The objective may contain endpoint terms, integral terms, or both. RAPTOR separates these into Mayer and Lagrange cost families.

Mayer Terms A Mayer term is a boundary function and is suitable for costs based on initial or final time, states, controls, or parameters. The brachistochrone problem minimizes final time using the following Mayer term:

Mayer Cost

```
f = phase.costs.mayer.add(@(inputData) inputData.finalTime);
f.functionType = 'linear';
```

Lagrange Terms A Lagrange term is a path function whose value is integrated over time by the transcription. Use a Lagrange term for running costs such as control effort, fuel consumption rate, or accumulated tracking error.

4.6.6 Function Properties and Derivatives

Calling `add` more than once on the same dynamics, constraint, or cost object defines that element as the sum of the registered functions. This supports modular formulations and allows linear and nonlinear contributions to be declared separately.

The object returned by `add` provides access to properties such as `functionType` and, where supported, manually supplied Jacobian values or sparsity patterns. Correctly identifying a linear contribution can avoid unnecessary numerical derivative approximation for that part of the model.

During the build process, RAPTOR evaluates user-defined functions on the initial guess and estimates their Jacobian sparsity patterns. The first estimate perturbs the inputs with `NaN` values to obtain a conservative dependency pattern. Some MATLAB or external functions do not accept `NaN` inputs; if this evaluation fails, RAPTOR falls back to regular finite differences around the initial guess. Because the fallback pattern depends on that guess, verify it carefully when a function changes behavior across different regions of the search space.

User-defined functions must also be compatible with the numerical differentiation method selected after the problem is built. Relevant restrictions are discussed in section 5.

4.7 Collocation Settings

Each phase must be discretized before the continuous problem can be transcribed. The selected polynomial family, node family, interpolant, and number of nodes determine the collocation grid and approximation used by RAPTOR. The implemented pseudospectral methods are described in the associated publications.

Collocation Settings

```
phase.collocation.interpolant = 'Lagrange';
phase.collocation.polynomials = 'Legendre';
phase.collocation.nodes = 'Gauss-Radau';
phase.collocation.nNodes = 49;
```

Increasing the number of nodes may reduce the mismatch between a converged collocation solution and its numerically propagated trajectory, but it also increases the size of the NLP. It does not correct an inconsistent formulation or a poor initial guess. Start with a moderate grid and use the validation results to decide whether refinement is necessary. For sufficiently smooth solutions, pseudospectral methods can exhibit spectral convergence and achieve high accuracy with a modest number of nodes [3].

4.8 Linkage Conditions

Linkage conditions relate the end of one phase to the beginning of another. Custom linkage conditions use the boundary-function convention described above. They can enforce continuity or model known jumps, staging events, and changes of state representation. Always verify the component order, units, and sign convention on both sides of a link.

The brachistochrone problem contains only one phase and requires no linkage condition. In examples/08-boostback2D/, the boostback, free-fall, and landing phases are connected using continuity conditions.

Linkage Conditions

```
ocp.linkage.continuity(boostback, freefall);
ocp.linkage.continuity(freefall, landing);
```

The `continuity` method enforces state and time continuity between the two phases. It also informs postprocessing that the phases form one continuous trajectory, so numerical propagation uses the final state of the first phase as the initial state of the second.

For more general relations, use the linkage object's `stack` method. Its syntax follows that of a boundary constraint, with the two phase objects supplied to identify the linked endpoints. The `continuity` method may also receive a conversion function that maps the final state of the first phase into the coordinates of the second; the same mapping is then used for both optimization and numerical propagation.

5 Configuring the Solver

After building the problem, configure its numerical behavior through `problem.config`. The configuration controls transcription, scaling, derivative estimation, the NLP solver, validation, and plotting.

Keeping numerical configuration separate from the continuous formulation makes it possible to compare methods without changing the model. Configure each stage independently: first select its method, then adjust the settings associated with that method.

5.1 Configuration Order

Assigning a method resets the settings for that stage to the defaults associated with the selected method. Therefore, always select the method before changing its settings. This order also allows MATLAB autocomplete to display the fields available for the active method. Reassigning the method later may discard previously entered method-specific values.

5.2 Scaling

Scaling improves numerical conditioning by bringing variables, constraints, and derivatives to more comparable magnitudes. It is particularly important when a formulation combines quantities with substantially different units or orders of magnitude.

The brachistochrone example applies two scaling methods in sequence:

1. projection on the bounds [4],
2. static Ruiz scaling [5].

Scaling Configuration

```
problem.config.scaling.method = ['BoundsProjection' 'RuizStatic'];
```

Scaling is not a substitute for a well-formulated problem. Use consistent physical units and realistic finite bounds where possible. When variables differ substantially in magnitude, begin with `BoundsProjection` or `Manual` scaling. If the solver converges slowly or exhibits oscillatory behavior, methods such as `RuizStatic` or `JacobianRowNormalization` may further improve the numerical conditioning.

Multiple scaling methods may be composed by assigning an ordered array of method names. RAPTOR builds and applies each method in sequence; the output of one method becomes the input to the next. The order is therefore significant, particularly for methods derived from function values, gradients, or Jacobians. Method-specific options are available through `problem.config.scaling.settings.methodName.(specificSetting)`.

5.3 Numerical Differentiation

The transcribed problem requires derivatives of its objective and constraints. When exact derivatives are not supplied, RAPTOR estimates them using the configured numerical differentiation method.

The example selects complex-step differentiation with a small step size. Unlike conventional finite differences, the complex-step method avoids subtractive cancellation and can therefore use very small perturbations for compatible functions.

Numerical Differentiation

```
problem.config.transcription.numericalJacobian = '
    ComplexStepDifferences';
problem.config.transcription.finiteStep = 1e-10;
```

Complex-step differentiation requires every user-defined function to preserve the imaginary perturbation. Explicit conversion to real values and non-analytic operations, such as MATLAB's `norm` function, break this requirement and invalidate the resulting derivatives. In such cases, use central differences instead and choose the step size carefully. The default central-difference step is `1e-5`; a step that is too small may amplify round-off error, while one that is too large may introduce truncation error.

5.4 NLP Solvers

Transcription converts the continuous problem into an NLP. Select an installed solver and set its convergence tolerance, iteration limit, and solver-specific options through the solver configuration.

The brachistochrone example selects SNOPT and sets its convergence tolerance to `1e-8`:

NLP Solver Configuration

```
problem.config.solver.method = 'Snopt';
problem.config.solver.tol = 1e-8;
```

Solver tolerances should be chosen together with discretization and validation accuracy. A tighter NLP tolerance does not necessarily produce a more accurate continuous trajectory and may increase run time or expose conditioning problems.

5.5 Solution Validation

Validation numerically propagates the recovered trajectory independently of the collocation equations. In the example, validation uses the variable-step Runge–Kutta integrator provided by MATLAB's `ode45` function.

Validation Method

```
problem.config.postprocessing.validation.method = 'RungeKutta45';
```

Treat validation as a separate acceptance step after solver convergence. Compare the propagated trajectory with the collocated solution and inspect constraint satisfaction before relying on the result. A small NLP residual does not by itself guarantee a small continuous-time error.

5.6 Postprocessing and Plotting

Postprocessing can generate plots of the primal solution, reconstructed dual quantities, and validation results. In the brachistochrone example, the `showPlots` flag controls each plot family.

Postprocessing Plots

```
problem.config.postprocessing.plotting.primal = showPlots;
problem.config.postprocessing.plotting.dual = showPlots;
problem.config.postprocessing.plotting.validation = showPlots;
```

RAPTOR reconstructs dual information using the covector mapping theorem [6]. The reconstructed costates and Hamiltonian can support an independent first-order optimality check based on Pontryagin's minimum principle. Plot configuration and custom plotting callbacks are described further in section 6.

6 Solving the Problem

Building and solving are separate operations. The build step checks the continuous problem definition and creates the transcribed numerical model. The resulting `problem` object is then configured before the selected NLP solver is called.

6.1 Building the Transcribed Problem

Building the Problem

```
problem = raptor.build(ocp);
```

The build process freezes the contents of the supplied optimal control problem object. Changes made to `ocp` after this call do not alter the built problem; rebuild the problem whenever the continuous formulation changes.

During the build, RAPTOR evaluates the registered user functions at test inputs and checks the consistency of dimensions, bounds, and other problem data. If a user function fails, RAPTOR first reports the underlying MATLAB error and then provides additional information about the context in which it occurred. Resolve all build errors before changing solver settings, because the NLP solver has not yet been called at this stage.

6.2 Running the NLP Solver

After configuring the built problem as described in section 5, call `solve`:

Solving the Problem

```
sol = problem.solve();
```

The selected NLP solver prints its iteration output in the MATLAB Command Window. Use this output to monitor feasibility, optimality, iteration count, and any numerical warnings during the solve. Always use the reported termination status to determine whether the solver claims convergence; the presence of a returned solution structure alone is not evidence of success.

6.3 Accessing the Solution

The returned solution structure organizes results by phase name. Depending on the enabled post-processing options, it contains primal variables and constraints, reconstructed dual variables and Hamiltonian data, and numerical propagation results.

Each phase contains `discretized`, `collocated`, and `propagated` fields. These fields present the phase data on the time grids associated with discretization, collocation, and numerical propagation. Select the field appropriate to the intended analysis rather than assuming that all arrays share the same number of columns.

Because the brachistochrone phase is named `descent`, its discretized time and first state component are accessed through `sol.descent`:

Solution Access

```
x = sol.descent.discretized.states(1, :);
t_disc = sol.descent.discretized.time;
```

The solution preserves the component order used in the continuous problem definition. Check array dimensions before postprocessing, particularly for multi-phase problems and single-component variables. When using a `Birkhoff` interpolant, the build process augments the control representation; inspect the returned control dimensions before indexing that field.

6.4 Plotting Results

6.4.1 Automatic Plot Metadata

RAPTOR can generate plots of time-dependent primal variables and constraints, reconstructed dual variables and the Hamiltonian, and the numerically propagated solution. Plot names, labels, and units are obtained from the `metadata` field of each phase. The brachistochrone example defines the following variable metadata:

Variable Metadata

```
% Metadata
phase.metadata.name.time = 'time';
phase.metadata.label.time = 't';
phase.metadata.unit.time = 's';
phase.metadata.name.states = ['horizontal position' 'vertical
   position' 'velocity'];
phase.metadata.label.states = ['x' 'y' 'v'];
phase.metadata.unit.states = ['m' 'm' 'm/s'];
phase.metadata.name.controls = 'inclination';
phase.metadata.label.controls = 'u';
phase.metadata.unit.controls = 'rad';
```

Constraints may define a label and unit in the same way. Their display name is taken from the name assigned when the constraint group is created.

Constraint Metadata

```
phase.constraints.path.stack('slope', -inf, aux.b);
phase.constraints.path.slope.label = '$dy/dx$';
```

6.4.2 Custom Plotting Callbacks

A custom plotting function can be called after postprocessing completes. The function receives the solution structure as its only input and may use it to generate problem-specific figures. The brachistochrone example registers its callback only when plotting is enabled:

Plotting Callback

```
if showPlots
    problem.config.postprocessing.plotting.auxiliaryFunction = @
        visualizeBrachistochroneSolution;
end
```

6.5 Diagnosing Problems

A failure may originate in the continuous formulation, its MATLAB implementation, the discretization, the initial guess, scaling, derivative estimation, or the external solver. Diagnose the earliest failing stage before changing multiple settings.

6.5.1 Build Errors

If the problem does not build, check function signatures, input and output dimensions, variable ordering, bounds, fixed boundary values, and option names. Reproduce the failing user-function call with representative data when possible. Solver tolerances and iteration limits are irrelevant until the build succeeds.

6.5.2 Failure to Converge

If the problem builds but the solver does not converge, check the following in order:

1. compare the implemented equations, units, signs, and variable ordering with the mathematical formulation;
2. confirm that bounds, fixed boundary values, and general boundary constraints are mutually consistent;
3. inspect the initial guess for bound violations, discontinuities, and unrealistic profiles;
4. evaluate user-defined functions and their derivatives at representative inputs;
5. review scaling and the magnitudes of variables, objective terms, and constraints;
6. solve on a moderate collocation grid before increasing the number of nodes;
7. inspect the solver termination status before changing tolerances or iteration limits.

6.5.3 Converged but Inaccurate Solutions

If the solver reports convergence but numerical validation is poor, inspect the propagated and collocated trajectories to identify where they diverge. Refine the collocation grid or revise the transcription settings and solve again. Tightening only the NLP tolerance may not reduce the continuous-time error.

6.5.4 Requesting Support

When requesting support, provide a minimal reproducible problem, the RAPTOR and MATLAB versions, the operating system, the selected solver and its version, complete build and solver output, all configuration values changed from their defaults, and the result of running a supplied example in the same environment.

References

[1] Andreas Wächter and Lorenz T. Biegler. "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming". en. In: *Mathematical Programming* 106.1 (Mar. 2006), pp. 25–57. ISSN: 0025-5610, 1436-4646. DOI: 10.1007/s10107-004-0559-y. URL: http://link.springer.com/10.1007/s10107-004-0559-y (visited on 03/25/2026).

[2] I. Michael Ross. *A primer on Pontryagin's principle in optimal control*. eng. Second edition. OCLC: 952791172. San Francisco: Collegiate Publisher, 2015. ISBN: 978-0-9843571-1-6.

[3] I. Michael Ross and Mark Karpenko. "A review of pseudospectral optimal control: From theory to flight". en. In: *Annual Reviews in Control* 36.2 (Dec. 2012), pp. 182–197. ISSN: 13675788. DOI: 10.1016/j.arcontrol.2012.09.002. URL: https://linkinghub.elsevier.com/retrieve/pii/S1367578812000375 (visited on 06/08/2026).

[4] John T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. en. Second. Society for Industrial and Applied Mathematics, Jan. 2010. ISBN: 978-0-89871-688-7 978-0-89871-857-7. DOI: 10.1137/1.9780898718577. URL: https://epubs.siam.org/doi/book/10.1137/1.9780898718577 (visited on 11/07/2025).

[5] Daniel Ruiz. "A scaling algorithm to equilibrate both rows and columns norms in matrices". en. In: (2001).

[6] I. Michael Ross and Fariba Fahroo. "A Pseudospectral Transformation of the Covectors of Optimal Control Systems". en. In: *IFAC Proceedings Volumes* 34.13 (Aug. 2001), pp. 543–548. ISSN: 14746670. DOI: 10.1016/S1474-6670(17)39048-1. URL: https://linkinghub.elsevier.com/retrieve/pii/S1474667017390481 (visited on 03/12/2026).